

15-march-2026

chai aur Express

Date _____
Page _____

Starting a new Javascript project.
command:

```
npm init -y
```

This generates a file
(package.json)

Then install Express

Always specify precisely: @version 4 or @version 5.

Because if you do it directly, you will get the very latest version. But there are difference between version 4 and version 5.

command:

```
npm i express
```

0-1 express-intro

```
const express = require('express')
```

This line imports the Express framework so you can start build.

(and type is commonJS (CJS) (old way))

```
import express from "express"
```

(This line also imports the express framework

But the type is module ["type": "module"]

it's ES Modules (ESM) is the modern Javascript standard used in both browsers and newer Node projects

assignment

What does `process.exit(0)` do?

In Node.js, `process.exit()` is a 'emergency stop' button for your application. When you call it, you are telling Node.js to terminate the process immediately.

Think of it like pulling the plug on a computer rather than letting it shut down naturally.

This method also takes one optional argument an **Exit Code**.

- `Process.exit(0)`

This means the program finished successfully. It's mean 'Everything went fine' signal to the operating system.

- `process.exit(1)`

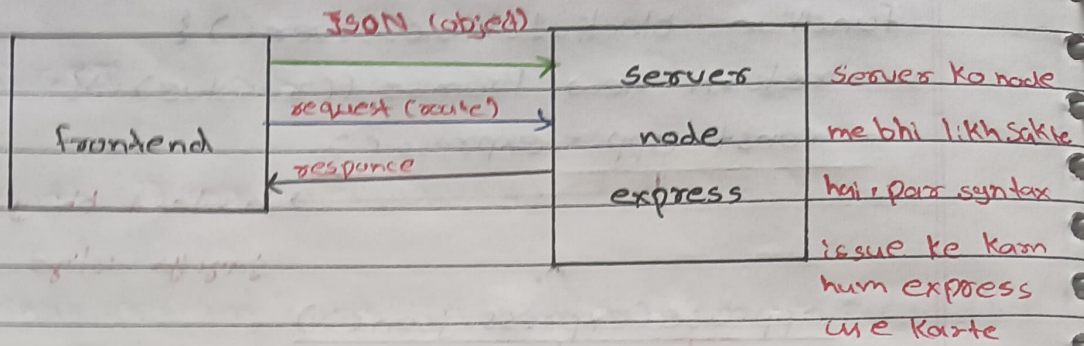
(or any non-zero number)

This tells the system the program crashed or stopped due to an error.

JSON

JSON stands for **JavaScript Object Notation**. It is a completely language-independent text format used to store and transport data.

Because JSON is easy for both humans and computers to read.



/register

/login

API endpoint

Can endpoint is usually a URL that points to a specific resource?

Why Express

Node.js has a build-in 'http' module for creating web servers, but:

- Requires manual handling of routing, parsing, headers.
- Verbose and repetitive code.

Express is a minimal web framework that provides:

- Easy routing (GET, POST, PUT, PATCH, DELETE)
- Middleware system for request / response processing
- Built-in request parsing (JSON, URL-encoded)
- Simplified the response method

~~Serialization Javascript Object~~

Serialization

Javascript object → JSON String (when sending response)

Deserialization

JSON String → Javascript object (available in request body)

Serialization vs Deserialization

Serialization:

Converting Javascript Object to JSON string

ex

```
const person = { name: "chaicode", age: new Date() }
```

```
JSON.stringify(person) →
```

```
'{"name": "chaicode",
```

```
"age": "2024-06-01T00:00:00.00z"}'
```

Deserialization

Converting JSON string to Javascript Object

ex.

```
'{"name": "chaicode"}' → { name: 'chaicode' }
```

Important

- Data lost during JSON serialization
- undefined values are omitted
- functions are omitted
- Date Objects becomes strings
- special types (map, set, RegExp) - use their type (RegExp is short for Regular Expression) is a special data type used for pattern matching within strings.

```
function block_1_basicServer() {  
  return new Promise((resolve) => {  
    const app = express()
```

→ frontend bhi yaha hi likha
jayege isliye

```
    app.use(express.json())
```

we ka matlab ←

middleware hai

→ instruction hai, ki json ko samajana
inbuilt method hai express ka json

Route: GET /menu (path) (handler)

```
app.get('/menu', (req, res) => {
```

direct response ← res.json({
bhejdo

```
  items: [
```

```
    'thali',
```

```
    'biryani'
```

```
  ]
```

```
})
```

```
})
```

→ Handler

- aap ne ek endpoint banaya 'get' or get ek
register karega handler ko '/menu' pe ya endpoint
pe,

- or jo res.json he vo ek serialized object
dega

rest.json ke do kam hote hai

1) aapke liye content type set karna
content-Type: 'application/json'

2) jo bhi aap bhej रहे ho use serialized kardo
matlab object ko , or

ye dono kam set me hi hota hai

Note: Endpoint / se start karney

ex '/start'

Sometimes request replace with '-' underscore

So hij use nahi hoti use replace kar dete hai

(- : res)

placeholder

Route: GET /search

Important (Demonstrates query parameters)

chaicode.com/cart?q=biryani&limit=5

- This is called as query param (parameters)
? (question mark) ke bad ki url me query start hoti hai.
- & this is connector, it allows you to chain multiple search rules together.
- limit=5 this is another query param

app.get('/search', (req, res) => {

const {q, limit} = req.query

req.query contains all query parameters as an object

res.json({

send data ←

query: q,

limit: limit || '10'

});

});

default to '10' if not provided

Note: If in url contains superman in the place of q you have to write same name the example,

chaicode.com/cart?superman=biryani&limit=5.

const {superman, limit} = req.query

Note! - you can use get post dono chalta hai, uska ko phosak nahi padta

OR

fancy words

⇒ Called callbacks as Handles upgrade version.

Route: GET /menu/:id
route / parameters (param) OR path parameters

```
app.get("/menu/:id", (req, res) => {
```

(agar upper id likha hai to niche bhi id likhni padegi)

```
  const {id} = req.params;
```

superman diya to superman likhna padega

req.params contains all path params

```
  res.json({
```

```
    item: id,
```

```
    price: 149, → Koi bhi ho de sakte
```

```
  });
```

ho, sab chalta.

```
});
```

Route: POST /order

Demonstrates POST request with request body

(post ka matlab db me kuchh to kuchh bhejna hi hai j)

(payload ⇒ data)

fancy word

```
app.post("/order", (req, res) => {
```

```
  const order = req.body
```

req.body contains the parsed JSON data sent in the request

```
  res.status(201).json({ → res.status ek variable
```

```
    status: "created",
```

object hai jo http ka

```
    order,
```

status set karata hai,

```
  },
```

or json serialized karke data send karoge

```
  },
```

Status 201 = created ('Successful resource creation')

Start Server & Test Router

Port 0 = OS assigns random available port

production me nahi karfehail, par localhost pe kar sakte

```
const server = app.listen(0, async () => {
```

```
  const port = server.address().port;
```

```
  const base = `http://127.0.0.1:${port}`;
```

↳ localhost

```
  try {
```

```
    const menuRes = await fetch(`${base}/menu`)
```

```
    const menuData = await menuRes.json() → parse karne
```

```
    console.log('GET / menu', JSON.stringify(menuData))
```

↓
menuData ek object hai

use string me convert

Karna fir

```
    console.log("++++ +++++ +++++ +++++ +++++ +++++")
```

```
    const searchRes = await fetch(`${base}/  
      search?q=biryani & limit=5`)
```

```
    const searchData = await searchRes.json()
```

```
    console.log('GET / search', JSON.stringify(searchData))
```

```
    console.log("++++ +++++ +++++ +++++ +++++ +++++")
```

```
    const menuItemRes = await fetch(`${base}/menu
```

(42'); string bhi
le sakte hai

```
    const menuItemData = await menuItemRes.json();
```

```
    console.log('POST / menu', JSON.stringify(menuItemData));
```

```
    console.log("++++ +++++ +++++ +++++ +++++ +++++")
```

difference yahi hai, object open hai

```
const orderRes = await fetch(`${base}/order`, {
  method: "POST", ← by default get hi heta hai.
  headers: {
    "Content-Type": "application/json",
    body: JSON.stringify({
      dish: "biryani",
      quantity: 2,
    })
  }
});
```

```
const orderData = await orderRes.json();
console.log("POST/Order", JSON.stringify(orderData));
console.log("++++++");
```

```
} catch (error) {
  console.log(error);
}
```

error handling

```
Server.close(() => {
  console.log("Blocked 1 served...");
  resolve();
});
```

server close karo

ek callback ke sat

```
});
```

jo task listing start ki thi use

```
});
```

band kardi, nahi sunam ab nahi

```
console.log(server);
```

```
});
```

Block 2: Response method in express

Different ways to send responses

```
function block-2-response () {  
  return new Promise ((resolve) => {  
    const app = express()
```

```
    app.get('/text', (req, res) => {  
      res.send("Hello from charcode");  
    });
```

res.send - sends various types (string, object, array)

```
    app.get("/json", (req, res) => {  
      res.json({  
        framework: "express",  
        version: "6.1.1",  
      });  
    });
```

res.json explicitly sends JSON response with correct content-type header

```
    app.get("/not-found", (req, res) => {  
      res.status(404).json({  
        error: "Page not found",  
      });  
    });
```

*res.status - sets HTTP status code
404 - Not found.*

Note - autoscaling group: usme aap Aus ke under aap machine laga sakte ho, agr aap ko 5 machine ki jarurat hai, s ka s laga sakte ho API handling karne ke lige, agr ek bhi down jati, ^{Data Page} ho to automatically ASG aap ko dusri machine up karke deta hai; or ASG ko health track se ye pata chalta hai

healthroute also called as heartbeat

health ki
jgh ping bhi
we kar sakte
ho

```
app.get("/health", (req, res) => {  
res.status res.sendStatus(200);  
});
```

res.sendStatus - sets status and sends status code as response

200 - OK (commonly used for health checks)

```
app.get("/old-menu", (req, res) => {  
// add entry point in DB to see how many users  
are still visiting old route  
res.redirect(301, "/new-menu");  
});
```

res.redirect() - redirects to another url

301 = Permanent redirect (good for SEO)

(So, you can track old route usage before redirecting)

```
app.get("xml", (req, res) => {  
res.type("application/xml").send("<dish><name>  
Biryani</name></dish>");  
});
```

res.type() - sets content-type header
useful for sending non-JSON response

```
app.get("/custom-headers", (req, res) => {
```

^{parameter} res.set("X-powered-By", "charcode");

^{start karna} res.set("X-Request-Id", "12345");

^{Best practices} res.json({

message: "Custom headers set",

});

}); res.set - sets custom response headers.

Note:

custom-headers,
CORS tracking, tracking insub me me to c'hal

Date
Page

```
app.get("/no-content", (req, res) => {  
  res.status(204).end(); res.status(204).end();  
});
```

res.end() - ends response without sending data
204 - No content (success but no response body)

```
const server = app.listen(0, async () => {  
  const port = server.address().port;  
  const base = `http://127.0.0.1: ${port}`;
```

```
try {
```

```
  // TODO
```

```
} catch (error) {
```

```
  console.log(error);
```

```
}
```

```
});
```

```
});
```

```
}
```

```
async function main() {
```

```
  await block1 - basicServer();
```

```
  await block2 - basicServer();
```

```
  process.exit(0);
```

```
}
```

```
main();
```

02 - routing

```
const express = require('express')
function block 1 - httpMethods() {
  return new Promise((resolve) => {
    const app = express()
    app.use(express.json())
    ↓ middleware
  })
}
```

```
const routes = [
  1: {
    id: 1,
    name: 'Dadar Andheri Express',
    direction: 'North',
  },
```

This is Database

```
2: {
  id: 2,
  name: 'Bandra - Kurla Shuttle',
  direction: 'East'
}
```

```
} let nextId = 3;
```

list all train

```
app.get('/routes', (req, res) => {
  res.json(Object.values(routes));
});
```

single route by id

```
app.get('/routes/:id', (req, res) => {
  const route route = routes[req.params.id]
```

or

```
const {id} = req.params
const route = routes[id]
```

```
const route = routes[req.params.id]
```

agar route mila hi nahi to

```
if (!route) return res.status(404).json({
  error: "No train on this id"});
res.json(route)
}
```

```
app.post('/routes', (req, res) => {
```

// no validation, no zod

(jo bhi data aasa cuki jach postal kar lo)

```
const newRoute = {id: nextid++, ...req.body}
```

```
routes[newRoute.id] = newRoute;
```

```
res.status(201).json(newRoute)
```

```
});
```

```
app.put("/routes/:id", (req, res) => {
```

(put me hamara id likhna, cuki bina put nahi likh sakte)

```
const id = req.params.id
```

routes ke under id nahi hai

```
if (!routes[id]) return res.status(404).json({
  error: "Something went wrong nahi
  bhajna h"});
```

```
routes[id] = {id: Number(id), ...req.body}
```

id pe data change krdo;

put me sara data bhajna krta hai

```
app.patch("/routes/:id", (req, res) => {
```

```
const id = req.params.id;
```

```
if (!routes[id]) {
```

```
return res.status(404).json({ error:
  "Route not found"});
```

```
}
```

// we keep the old data and overwrite only what's in req body

```
const updatedRoute = {  
  ...routes[id], ...req.body, '}'
```

// save it back to your data store

```
routes[id] = updateRoute;
```

// send back the updated object

```
res.json({  
  message: "updated successfully",  
  data: routes[id],  
});
```

```
app.delete("/routes/:id", (req, res) => {
```

```
  const id = req.params.id
```

```
  if (!routes[id]) return res.status(404).json(  
    {error: "Something went wrong nahi  
    bhejna hai"});
```

```
  delete routes[id]
```

```
  res.status(201).end()
```

↓
delete bhi ek operation hai.
or vo bhi successful hoga

(2 ke hi range me status code noticate)

```
const server = app.listen(0, async () => {
```

```
  const port = server.address().port;
```

```
  const base = `http://127.0.0.1:${port}`;
```

```
try {  
  const listRes = await fetch(`${base}/routes`);  
  const listData = await listRes.json();  
  
  const creatRes = await fetch(`${base}/routes`,  
    method: "post",  
    headers: {  
      "Content-Type": "application/json",  
    },  
    body: JSON.stringify({  
      name: "Colaba-Worli",  
      direction: "South",  
    })  
  );  
  
  const created = await creatRes.json();  
} catch (error) {  
  console.log(error);  
}
```

Important

```
app.get('/files/*filepath', (req, res) => {  
  const filepath = req.params.filepath  
  res.json({ filepath, type: "wildCard" })  
})
```

- * filepath matlab /files ke baad kuchh bhi aa sakta hai jese /files/docs/readme.txt in subko catch karne ke liye
- filepath ka datatype array hai, data ume hi milenge
- wildCard matlab kuchh bhi jese /files/assets/styles.css

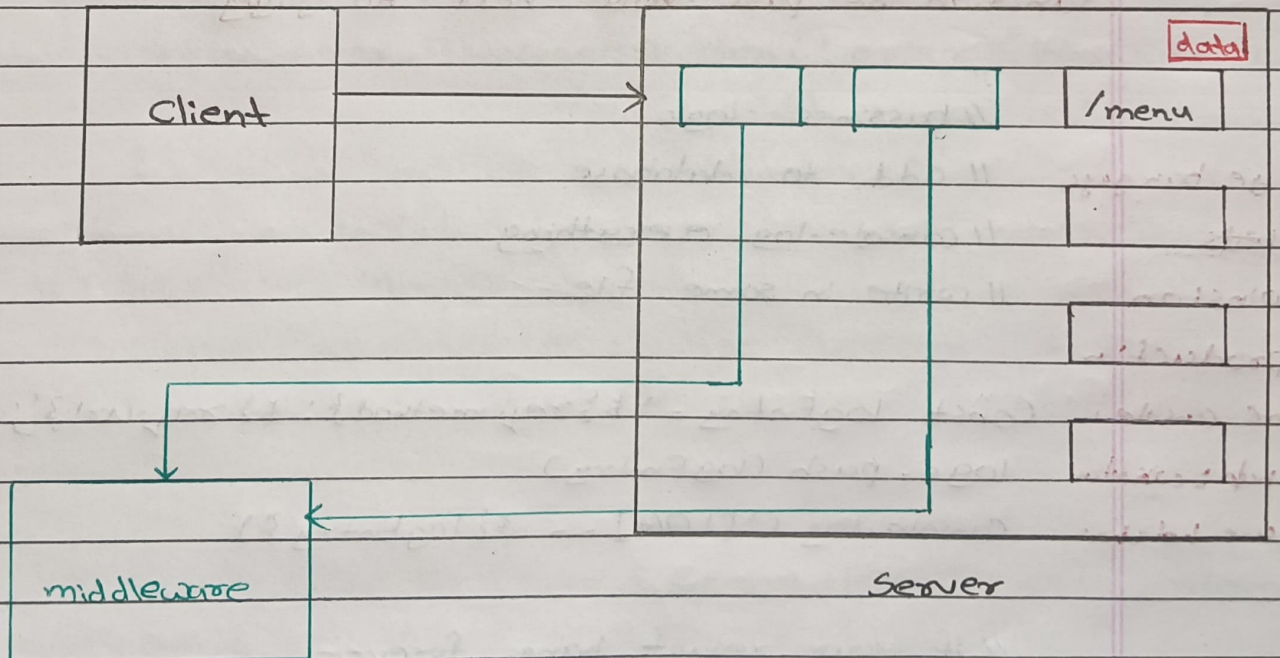
app

- route("/schedule")
- get((req, res) => {})
- post((req, res) => {})
- put((req, res) => {})
- delete((req, res) => {})

```
app.use("/api", (req, res) => {
  // its a prefetch match
})
```

Subse pehle app.use chalta hai! In sabhi se pehle /api se start hoga to inke pass se hi hote hain

03 - middleware



Note:-

limit of middle ware 35 lakh.

next tabhi use hota hai, jab aap middleware likh rhe ho.

Date

Page

```
const express = require('express')
```

```
function block-1-httpMethods() {
```

```
  return new Promise((resolve) => {
```

```
    const App = express()
```

```
    app.use(express.json())
```

↳ Kisi bhi request ke ye route ke pass ja rhae, to milke jana

```
    const logs = []
```

```
    // request logger
```

(Koi request aayegi, ya jayegi yaha entry karke jayegi)

iske pass 4 parameters hote hai, req, res, next or error

error
option
not
hai

```
    app.use((req, res, next) => {
```

middleware aap ki sari request ko rokega, or is function se pass hoke next ho jayegi

```
      // business logic
```

ye ban gae, // add to database

logs // console.log everything

winston, // write in some file

production

we console, const logEntry = `\${req.method}: \${req.url}`;

logs.push(logEntry)

console.log(`[LOG] -- \${logEntry}`)

```
    // if your request hangs forever
```

next() → (smj jana aap tab next() likh
na bhul gaye ho tab)

```
  })
```

```
app.use((req, res, next) => {
  req.startTime = Date.now();
```

```
  res.on('finish', () => {
    const duration = Date.now() - req.startTime;
    console.log('[Timer] - ${req.method} - 
    ${req.url} took ${duration}ms')
  })
```

• On se req or res pe event listener laga sakte ho

Custom middleware

```
function authMe (req, res, next) {
  const token = req.headers['x-auth-token']
```

```
  if (!token) {
    return res.status(401).json({ error: "No Token, 
    please login" })
  }
```

```
  if (token !== 'secret-chalcode') {
    return res.status(403).json({ error: "Invalid 
    token" })
  }
```

```
  // token → extract form token → userID, email, Student admin
  req.user = { id: 1, name: "Dipali", role: "Student" };
  next();
}
```

```
app.get('/profile', authMe, another, oneMore, () => {})
```

express ka kam hai ki profile route pe aaga to ensub ko call
karana jis order me diya hai, usi order me

reusable function

```
function getRole (role) {  
  return (req, res, next) => {  
    if (!req.user || req.user.role !== role) {  
      return res.status(403).json({ error:   
        `Role ${role} required` })  
    }  
  }  
}
```

```
app.get('/profile', authMe, getRole('admin')()) => {}  
app.get('/profile', authMe, getRole('teacher')()) => {}  
app.get('/profile', authMe, getRole('student')()) => {}
```

Pehle authMe ne request.user dal diya, matlab req.user ke under role dalne ki jimmadari authMe ki hai.
fir check karna ki aap kisi bhi role ko admin only, teacher only, student only bana sakte ho

Custom ratelimit function

```
function ratelimit (maxRequest) {  
  let count = 0;  
  
  return (req, res, next) => {  
    count++
```

```

if (count > maxRequest) {
  return res.status(429).json({error: "Too many request, please try after some time"});
}
next()
}
}

```

```

const limitedEndPoint = rateLimit(3)

```

```

app.get('/limited', limitedEndPoint, (req, res, ) => { })

```

Important

use tab use karna jab in hamare pass build in middleware ho, (adhar ka code).

agar 3rd party package ka huva tab bhi;

yaha pe bhi user ko direct rok sakte hai

```

app.use(express.json({limit: '50kb'}))

```

Zero trust architecture

"Never trust, always verify"

Security ki sabi zimmedari backend pe hai.

URL Encoding (Percent-encoding)

is a mechanism for translating 'unsafe' characters in a url into a format that can be safely transmitted over the internet.

Ex

Dipali 9020 Sharma

└─ it's a space.

```

app.use(express.urlencoded({extended: true, limit: '50kb'}))

```